Frontend Assignment –III

Theory part – Concepts of JS

Parth Goswami

NC-36 or 22515000096

1. Difference Between Frontend and Backend JavaScript

JavaScript can be used for both frontend and backend development, but their purposes are different.

Frontend JavaScript runs inside the user's web browser and is responsible for creating interactive user interfaces. It controls webpage elements, animations, form validation, dropdown menus, buttons, image sliders, and other features that users can directly see and interact with.

Backend JavaScript runs on a server using environments such as Node.js. It is responsible for handling server-side operations such as processing requests, interacting with databases, managing user authentication, storing data, and performing business logic.

The main difference is that frontend JavaScript deals with the presentation layer of an application, while backend JavaScript handles data processing and server-side functionality.

For example, when a user submits a registration form, frontend JavaScript validates the input, and backend JavaScript stores the data in a database.

2. What is DOM?

DOM stands for **Document Object Model**. It is a programming interface that represents an HTML or XML document as a hierarchical tree structure. Every element, attribute, and piece of text in a webpage becomes an object that JavaScript can access and manipulate.

When a webpage loads, the browser creates the DOM automatically. Through the DOM, JavaScript can change the content, structure, and style of a webpage without reloading it. This makes web pages interactive and dynamic.

For example, JavaScript can use the DOM to change the text of a paragraph, modify colors, create new elements, remove existing elements, or respond to user actions such as clicks and keyboard

inputs. In modern web development, the DOM plays a crucial role because it serves as a bridge between HTML documents and JavaScript. Without the DOM, JavaScript would not be able to access or modify webpage content dynamically.

3. What is an Event in JavaScript?

An **event** is an action or occurrence that happens in the browser and can be detected by JavaScript. Events allow web pages to respond to user interactions and other browser activities.

Examples of common events include clicking a button, pressing a keyboard key, moving the mouse, submitting a form, loading a webpage, and resizing a browser window. These events provide a way for developers to create interactive web applications.

Events are generally categorized into mouse events, keyboard events, form events, window events, and touch events. The event object contains detailed information about the event, such as the target element, mouse position, and key pressed.

Events are an essential part of JavaScript because they enable communication between users and web applications, making websites dynamic and interactive.

4. What is Event Loop in JavaScript?

The **Event Loop** is a fundamental mechanism that allows JavaScript to perform asynchronous operations efficiently. JavaScript is a single-threaded language, meaning it can execute only one task at a time. However, it can still handle multiple operations simultaneously because of the event loop.

The event loop continuously monitors two important components: the **Call Stack** and the **Callback Queue**. The call stack contains functions currently being executed, while the callback queue stores functions waiting to be executed.

When asynchronous operations such as `setTimeout()`, API requests, or file reading tasks are initiated, they are handled by browser APIs or Node.js APIs instead of the main thread. Once these operations are completed, their callback functions are placed in the callback queue.

The event loop is a key concept behind JavaScript's asynchronous programming model and is crucial for modern web development.

5. What is Hoisting in JavaScript?

Hoisting is JavaScript's default behavior of moving declarations to the top of their scope before code execution. During the compilation phase, JavaScript creates memory for variables and functions before executing the program.

In the case of variables declared with `var`, the declaration is hoisted but the assignment is not. Therefore, accessing a variable before its assignment results in the value `undefined`.

Function declarations are fully hoisted, which means they can be called even before they appear in the code. However, function expressions are not hoisted in the same way.

Variables declared using `let` and `const` are also hoisted, but they remain in a **Temporal Dead Zone (TDZ)** until their declaration is encountered. Attempting to access them before their declaration causes an error.

6. What is Event Bubbling?

Event Bubbling is the process through which an event propagates from the target element upward to its parent elements in the DOM hierarchy.

When an event occurs on a nested element, such as a button inside a div, the event is first handled by the target element and then bubbles upward through its ancestors until it reaches the document object.

For example, if a button is placed inside a div and both elements have click event handlers, clicking the button will first execute the button's event handler and then the div's handler. This behavior occurs because of event bubbling.

Event bubbling is the default event propagation mechanism in JavaScript. It is useful because developers can manage events at higher levels instead of attaching separate listeners to multiple elements. This technique is known as **event delegation** and improves performance.

7. Difference Between Named Function, Anonymous Function and Arrow Function

Functions are reusable blocks of code in JavaScript, and they can be defined in different ways.

A **Named Function** is a function that has its own identifier or name. It can be called directly using that name and is often easier to debug because the function name appears in error messages.

An **Anonymous Function** is a function without a name. It is usually assigned to a variable or passed as an argument to another function. Anonymous functions are commonly used as callbacks.

An **Arrow Function** was introduced in ES6 as a shorter syntax for writing functions. It uses the `=>` operator. Arrow functions are concise and automatically inherit the `this` value from their surrounding scope.

8. Explain for-of and for-in Loop

JavaScript provides several looping mechanisms, including **for-of** and **for-in** loops.

The **for-of loop** is used to iterate over the values of iterable objects such as arrays, strings, maps, and sets. It directly returns the values stored in a collection. This makes it easy to access elements without dealing with indexes.

The **for-in loop** is used to iterate over the enumerable properties or keys of an object. When applied to an array, it returns the array indexes rather than the actual values.

For example, when iterating through an array of student names, for-of returns the names themselves, while for-in returns their index positions.

Choosing between these loops depends on whether the developer needs values or property keys. Understanding both loops is important for writing efficient JavaScript programs.

9. Explain Any Five Array Methods

Arrays are one of the most commonly used data structures in JavaScript. Several built-in methods are available for performing operations on arrays.

1. push()

The `push()` method adds one or more elements to the end of an array and returns the new length of the array.

2. pop()

The `pop()` method removes the last element from an array and returns the removed element. It modifies the original array.

3. shift()

The `shift()` method removes the first element from an array and shifts all remaining elements one position to the left.

4. unshift()

The `unshift()` method adds one or more elements to the beginning of an array and returns the new length of the array.

5. map()

The `map()` method creates a new array by applying a function to each element of the original array. It does not modify the original array.

These methods help developers efficiently add, remove, and transform data within arrays. Since arrays are widely used in JavaScript applications, understanding these methods is essential for effective programming.

10. What is a Callback Function?

A **callback function** is a function that is passed as an argument to another function and is executed after a specific task or event has been completed.

Callbacks are commonly used in JavaScript because many operations take time to finish. Examples include loading data from a server, reading files, processing user input, or waiting for timers. Instead of blocking execution, JavaScript continues running other code and executes the callback when the operation is complete.